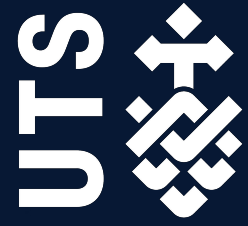


Assignment 2:

Data Analysis Report



36103: Statistical Thinking for Data Science

Submitted: May 10, 2026

Group Number: 5

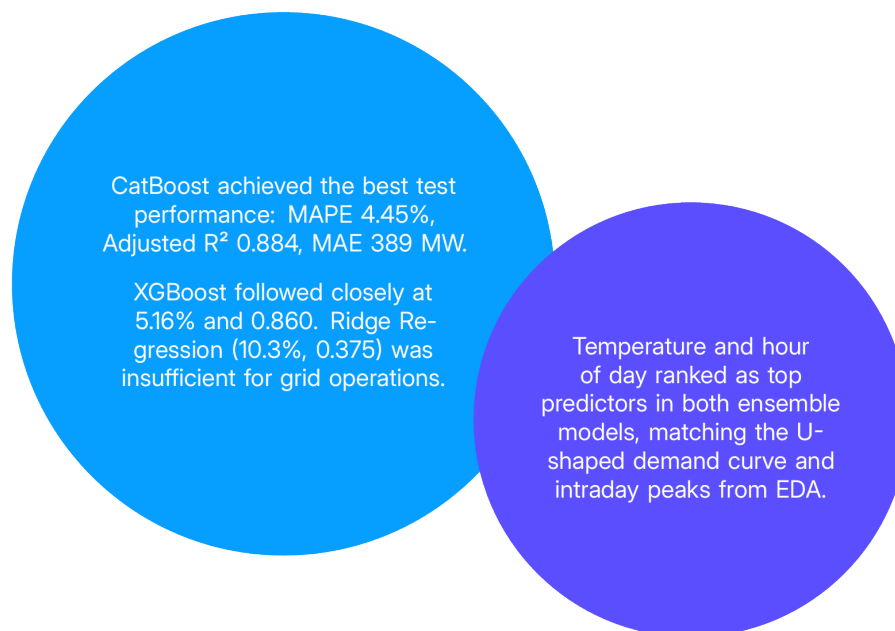
Yash Bankar	(25717428)
Preran Apinakoppa	(26583524)
Bhavya Subhash Chandar	(25599679)
Rohit Khanna	(25669108)
Murphy Fang	(26211831)

Executive Summary

Electricity demand in NSW varies with weather, time, and season. During peak periods, wholesale prices can reach \$17,500/MWh, making accurate forecasts important for grid stability and cost management.

Exploratory analysis showed demand is lowest in mild conditions, rises during extreme hot or cold weather, and differs between weekdays and weekends. Rooftop solar reduces midday demand but causes a sharp evening increase.

Analysing NSW demand (2022-2026), this study developed a 30-minute interval forecasting pipeline. CatBoost emerged as the optimal model, effectively capturing non-linear climatic and temporal drivers to support grid stability and market pricing strategies.



Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Rationale	2
1.3	Project Aims and Objectives	2
2	Methodology	3
2.1	Dataset	4
2.1.1	Data Acquisition and Cleaning	4
2.1.2	Feature Scaling	4
2.2	Technologies Used	4
2.3	Models	5
2.4	Grid Search	5
2.5	Metrics	5
3	Results	6
3.1	Key Findings	7
3.2	In-depth Results	7
3.2.1	Can an ensemble regression model predict 30-minute NSW grid demand with MAPE below 5%?	7
3.2.2	Model Interpretation and Feature Importance	8
4	Conclusion	9
4.1	Conclusion	10
4.2	Project Limitations and Caveats	10
4.3	Stakeholder Analysis and Project Outcomes	10
	References	11
	Appendices	13
A	Exploratory Data Analysis	14
A.1	General Exploration	14
A.2	Does temperature have a non-linear relationship with NSW grid demand across 2°C bins? . . .	14
A.3	Do mean hourly demand profiles differ between weekdays, weekends, and public holidays? . .	15
B	Modelling Code	17
B.1	Data Preparation	17
B.2	Modelling	21

List of Tables

1	Data Collation and Source Rationale	4
2	Model Selection and Theoretical Rationale	5
3	Chosen Performance Metrics	5
4	Model Performance Comparison	7

List of Figures

1	Target feature distribution	4
2	QQ-plots. All models show reasonable normality in the central quantile range with heavier tails at extremes, consistent with demand spikes during extreme weather events.	7
3	Feature Importance Comparison across Models	8
4	Multicollinearity Matrix on Raw Data	14
5	Mean NSW grid electricity demand across temperature bins.	15
6	Mean NSW intraday electricity demand by day type.	15
7	Impact of rooftop solar on NSW grid demand (the "duck curve").	16

Introduction

1 Introduction

1.1 Problem Statement

Electricity supply and demand must balance every 30 minutes. When they don't, grid operators absorb the cost. NSW wholesale prices can reach \$17,500/MWh under the market cap that applied through mid-2025 (AER, 2025b).

1.2 Rationale

Serving 11 million customers, the NEM relies on NSW as its primary generator (AER, 2026). NSW's 7.5 GW of rooftop solar (the highest globally per capita) creates a "duck curve" effect: suppressing midday demand before requiring a massive evening ramp from dispatchable generators (CEC, 2025; Green.com.au, 2026). This volatility drove 2024 wholesale prices up 43% to \$150/MWh, making accurate forecasting critical to reducing costs (AER, 2025a).

1.3 Project Aims and Objectives

The aim of this project was to determine whether machine learning models trained on historical half-hourly NSW data can predict grid electricity demand accurately enough to support operational dispatch planning through three research questions:

1. Does temperature have a non-linear relationship with NSW grid demand across 2°C bins from 5°C to 43°C?
2. Do mean hourly demand profiles differ between weekdays, weekends, and public holidays in NSW?
3. Can a regression model predict 30-minute NSW grid demand with a MAPE below 5% on test data?

To address these, we merged AEMO market data with environmental and economic indicators. We compared Ridge (baseline), XGBoost, and CatBoost models using a 60:20:20 chronological split, evaluated via RMSE, MAE, MAPE, and Adjusted R^2 .

Methodology

2 Methodology

2.1 Dataset

2.1.1 Data Acquisition and Cleaning

While raw AEMO market data was high-frequency, it lacked environmental and economic features needed for predictive modelling, which were synthetically merged using time-stamps as primary keys. Cleaning involved linear interpolation of missing observations, removing high multi-collinear features and frequency alignments. Please refer to Appendix Section A for a comprehensive **EDA**.

Table 1. Data Collation and Source Rationale

Data Source	Description	Methodology Rationale
AEMO (Market Data)	Half-hourly TOTAL_DEMAND and RRP (Regional Reference Price) for the NSW1 region.	Established the target variable and allowed for the analysis of price-demand elasticity during peak events.
AEMO (Solar Data)	Estimated ROOFTOP_SOLAR_MW generation across NSW.	Critical for deriving GROSS_DEMAND_MW, accounting for "behind-the-meter" generation that masks true load.
Bureau of Meteorology	Half-hourly temperature, humidity, wind speed, and cloud cover data for Sydney.	Included to model the primary driver of demand: the non-linear relationship between ambient temperature and HVAC load.
Economic Indicators	Monthly Cash Rate (RBA), Household Spend (ABS), and Consumer Confidence indices (ANZ-Roy Morgan Australian).	Integrated to capture macro-trends and shifts in baseline energy consumption patterns over the 4-year study period.

2.1.2 Feature Scaling

Feature engineering generated cyclical temporal encodings (Sine/Cosine transforms) to capture seasonal and daily periodicity. Data was standardised via One-Hot Encoding for categorical variables and Standard Scaling for numerical features. Detailed visualisations are provided in Appendix Section A.

2.2 Technologies Used

Developed in a Python 3.11 virtual environment, BeautifulSoup for web scraping, Pandas for data-frames, Cat-Boost and XGBoost libraries, scikit-learn, and Vegas-Altair for visualisation. Seeding was set to 42 for consistency and reproducibility. Resulted in 70,176 rows across 28 features. Chronologically split in 60:20:20 provided 42105:14035:14036 rows for training, validation and testing respectively.

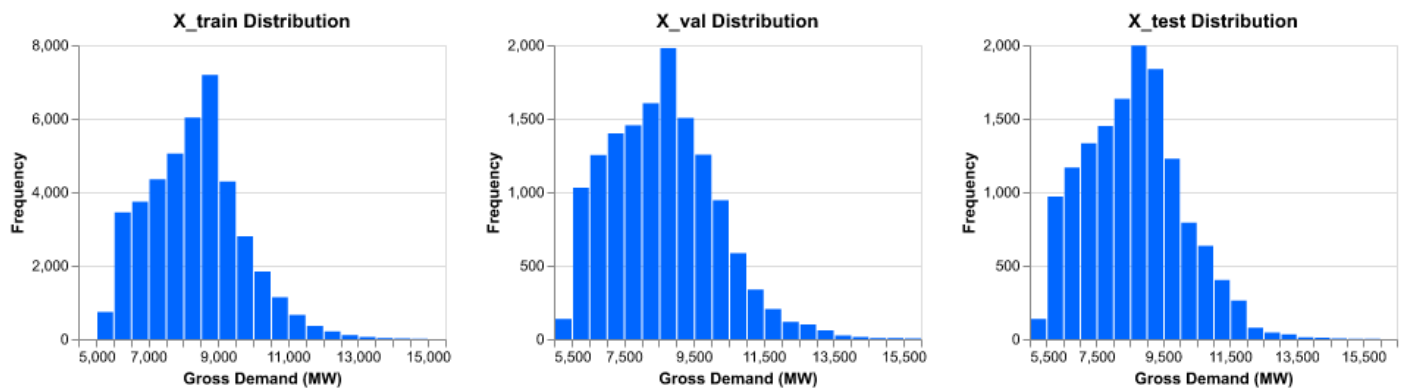


Figure 1. Target feature distribution

2.3 Models

Ridge Regression established a baseline using L2 regularisation to manage correlated weather predictors. XGBoost and CatBoost (GBDT) were implemented to capture non-linear relationships via sequential gradient descent. Hyperparameters were optimised via Grid Search with k -fold Cross-Validation.

Table 2. Model Selection and Theoretical Rationale

Model	Description	Methodology Rationale
Ridge	Linear model with L2 regularisation to penalise large coefficients.	Established a baseline to quantify non-linearity and manage multicollinearity among correlated weather predictors.
XGBoost	Scalable GBDT library designed for high computational efficiency.	Selected to address non-linear relationships in the data through sequential tree-building and dual regularisation (L1 and L2) to control model complexity.
CatBoost	Advanced GBDT algorithm that utilises symmetric trees and ordered boosting to reduce prediction shift and handle categorical features.	Chosen for its superior ability to generalise on unseen data and its efficient handling of categorical features (such as 'Season' and 'Holiday') without extensive manual encoding.

2.4 Grid Search

Hyperparameter optimisation is performed via Grid Search with k -fold Cross-Validation. Through exhaustive predefined parameter grids, we identified configurations that minimised generalisation error of each model.

2.5 Metrics

Table 3. Chosen Performance Metrics

Metric	Description and Purpose	Formula
MAE	Average absolute error in MW, operationally interpretable.	$\frac{1}{n} \sum y_i - \hat{y}_i $
MAPE	Scale-independent accuracy across demand magnitudes.	$\frac{1}{n} \sum \left \frac{y_i - \hat{y}_i}{y_i} \right \times 100$
RMSE	Penalises large outliers in peak demand periods.	$\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$
Adjusted R ²	Explained variance penalised for predictor count.	$1 - \frac{(1-R^2)(n-1)}{n-p-1}$

Model performance was evaluated using a hold-out test set to ensure generalisability. The following Table 3 above illustrates the chosen metrics.

Results

3 Results

3.1 Key Findings

CatBoost achieved the best performance (MAPE 4.47%), significantly outperforming the Ridge baseline (MAPE 10.31%). This disparity confirms the fundamentally non-linear nature of NSW electricity demand.

3.2 In-depth Results

3.2.1 Can an ensemble regression model predict 30-minute NSW grid demand with MAPE below 5%?

Table 4. Model Performance Comparison

Model	RMSE (MW)			MAE (MW)			MAPE (%)			Adj R ²		
	Train	Val	Test	Train	Val	Test	Train	Val	Test	Train	Val	Test
Linear Regression	999.66	1202.67	1163.85	792.86	943.75	907.86	9.61	10.70	10.29	0.46	0.39	0.37
XGBoost	301.27	576.66	550.99	221.96	460.53	438.33	2.60	5.54	5.16	0.95	0.86	0.86
CatBoost	314.68	502.46	500.33	230.44	391.92	389.00	2.71	4.66	4.45	0.95	0.89	0.88

Three models were compared on held-out test data (Table 4). Ridge Regression failed to capture non-linear patterns (MAPE 10.31%, $R^2 = 0.389$). XGBoost improved but showed overfitting (MAPE 5.83%, $R^2 = 0.836$). CatBoost generalised best across all metrics, meeting the sub-5% threshold.

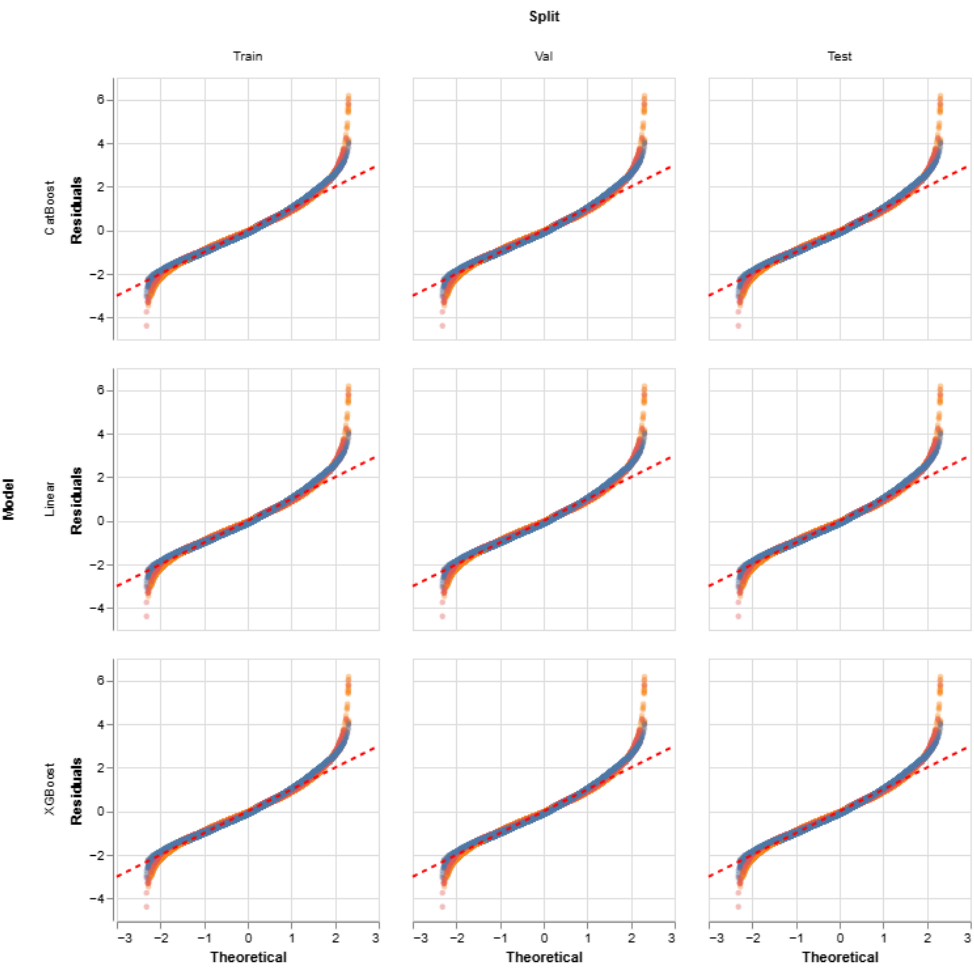


Figure 2. QQ-plots. All models show reasonable normality in the central quantile range with heavier tails at extremes, consistent with demand spikes during extreme weather events.

Regression assumptions were verified via QQ-plots (shown in Figure 2).

3.2.2 Model Interpretation and Feature Importance

Feature importance comparisons (Figure 3) indicate that while the Ridge baseline relied on temperature, CatBoost and XGBoost prioritised temporal and environmental drivers. This confirms the boosted models learned solar patterns through proxies, successfully compensating for the removed rooftop solar feature.

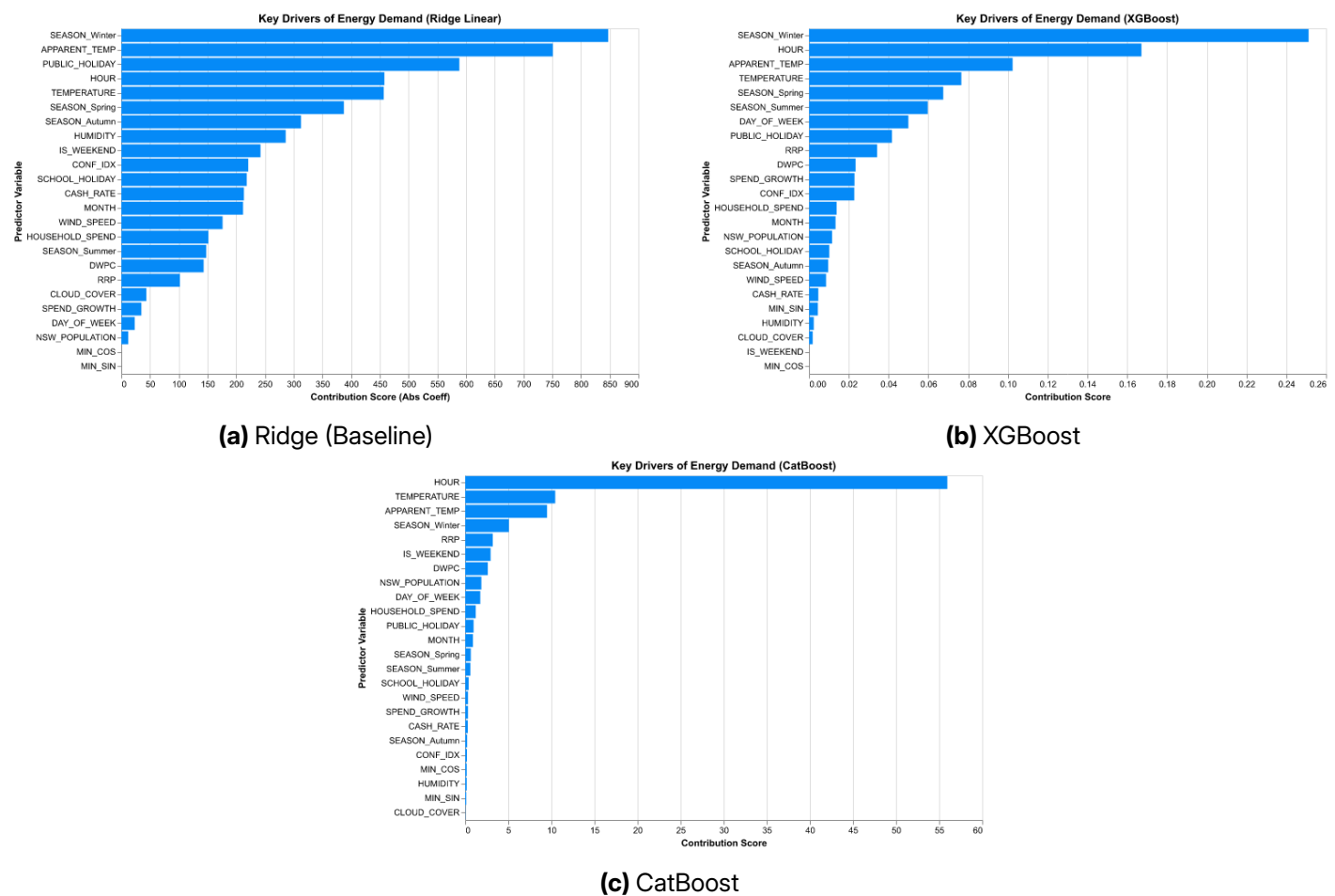


Figure 3. Feature Importance Comparison across Models

Conversely, the marginal impact of the **Cash Rate** suggest that while economic indicators provide macro-context, they are less critical for short-term 30-minute interval forecasting than environmental and temporal features.

Conclusion

4 Conclusion

4.1 Conclusion

CatBoost achieved the sub-5% target (MAPE 4.47%), outperforming XGBoost and the Ridge baseline. This success confirms that gradient-boosted ensembles effectively capture the non-linear relationship between temperature, temporal cycles, and demand. However, with an MAE of ~ 400 MW, the model remains a decision-support tool rather than a replacement for operator judgment, as errors of this magnitude represent significant financial risk at peak wholesale prices.

4.2 Project Limitations and Caveats

The model is fixed to a 2022-2026 training window with no self-update mechanism. Accuracy will drift as the grid changes. Urban areas also dominate the dataset, so regional and remote NSW performance is likely worse than the headline numbers suggest.

Remote and Aboriginal communities are underrepresented in AEMO grid data. A model trained mainly on urban demand risks reinforcing unequal energy distribution if applied broadly. Any deployment should respect Indigenous Data Sovereignty principles and involve community engagement before rollout.

4.3 Stakeholder Analysis and Project Outcomes

The deliverable is a CatBoost pipeline returning 30-minute demand forecasts from weather, calendar, and economic inputs. AEMO operators benefit most, in that the better forecasts mean less reliance on expensive last-minute generation. Next steps: live AEMO data integration, retraining cadence, and testing sequential models like LSTM for extreme weather periods.

References

References

- Australian Energy Regulator. (2025a). *Wholesale markets quarterly - q4 2024* (tech. rep.) (Accessed: 2026-05-08). Australian Energy Regulator. <https://www.aer.gov.au/publications/reports/performance/wholesale-markets-quarterly-q4-2024>
- Australian Energy Regulator. (2025b). *State of the energy market 2025* (tech. rep.) (Accessed: 2026-05-08). Australian Energy Regulator. <https://www.aer.gov.au/publications/reports/performance/state-energy-market-2025>
- Australian Energy Regulator. (2026). Our role [Accessed: 2026-05-08].
- Clean Energy Council. (2025). Australia powers ahead on rooftop solar as nation set to achieve 2030 rooftop target - new report finds [Accessed: 2026-05-08].
- Green.com.au. (2026). Solar statistics 2026 [Accessed: 2026-05-08].

Appendices

Appendices

A Exploratory Data Analysis

The objective of this exploratory phase was to move beyond the raw complexity to identify the primary drivers of Gross Energy Demand. In light of the data science project cycle, this EDA served two critical functions: **Feature Selection** and **Operational Insights**.

A.1 General Exploration

Observing multicollinearity such as through a correlation heatmap is critical because it identifies highly redundant predictors that can destabilise the model, inflate the variance of coefficient estimates, and make it impossible to determine the individual mathematical contribution of each variable to the target output.

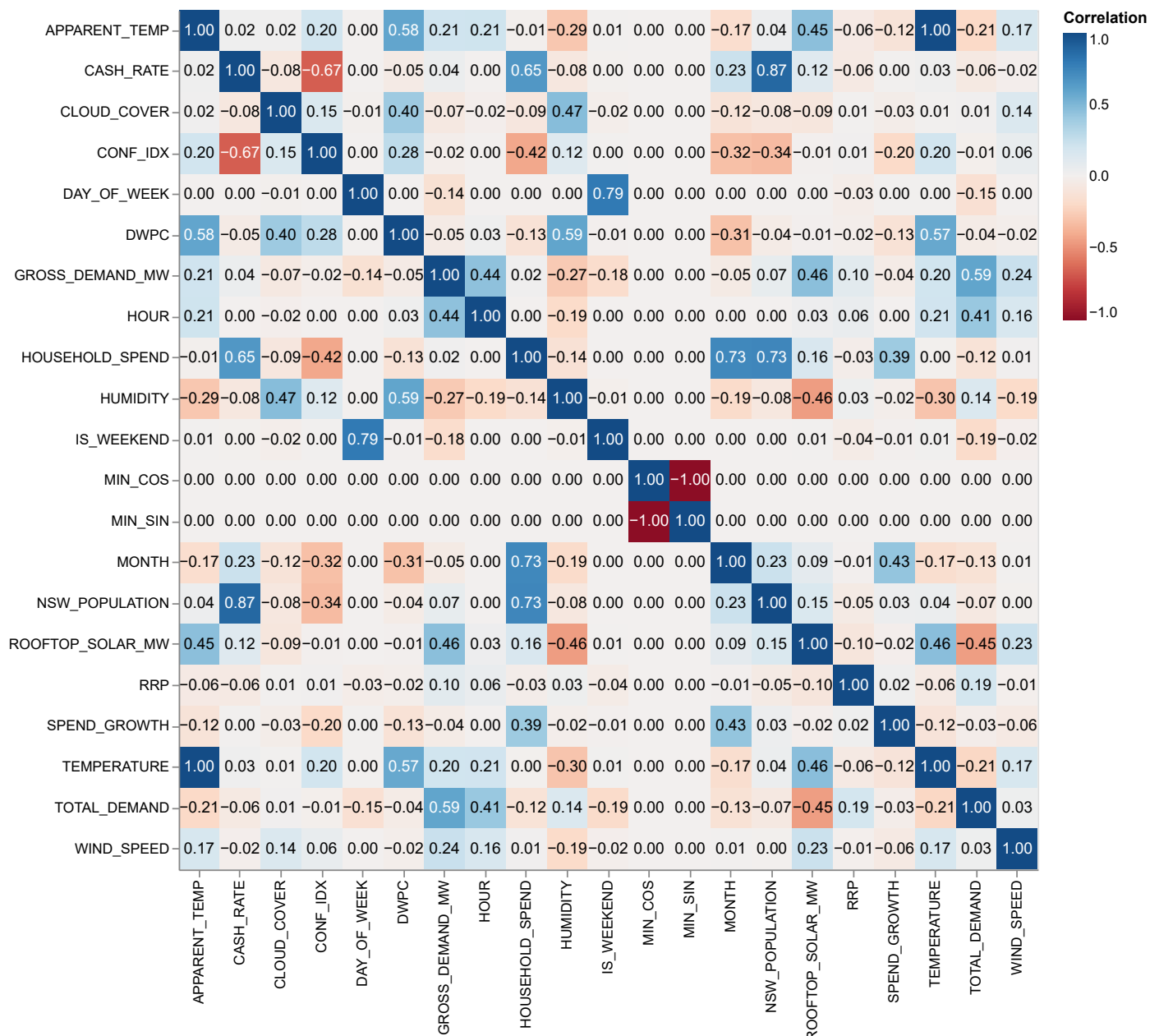


Figure 4. Multicollinearity Matrix on Raw Data

A.2 Does temperature have a non-linear relationship with NSW grid demand across 2°C bins?

A clear U-shaped relationship was observed between temperature and mean grid demand (Figure 5). Demand was lowest near 21°C and rose sharply at both cold and hot extremes, directly ruling out linear modelling.

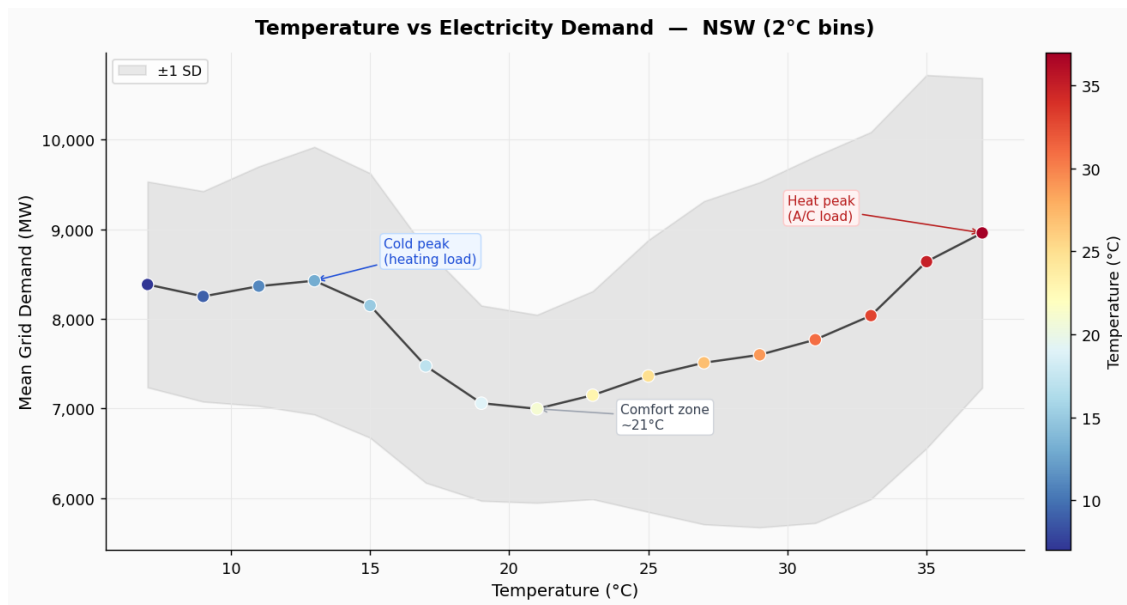


Figure 5. Mean NSW grid electricity demand (MW) across 2°C temperature bins (5°C-43°C). A U-shaped pattern is evident, with lowest demand near the comfort zone (~21°C, ~7,000 MW), rising at cold extremes due to heating load and hot extremes due to air-conditioning. Shaded band represents ± 1 SD.

A.3 Do mean hourly demand profiles differ between weekdays, weekends, and public holidays?

Weekday demand showed a morning commercial peak at 8 AM (~8,200 MW) and an evening residential peak at 7 PM (~9,500 MW). Weekend and holiday profiles were 1,000-1,500 MW lower with no morning peak (Figure 6). An unexpected finding was a pronounced midday dip across all day types driven by rooftop solar, forming the classic “duck curve” pattern (Figure 7).

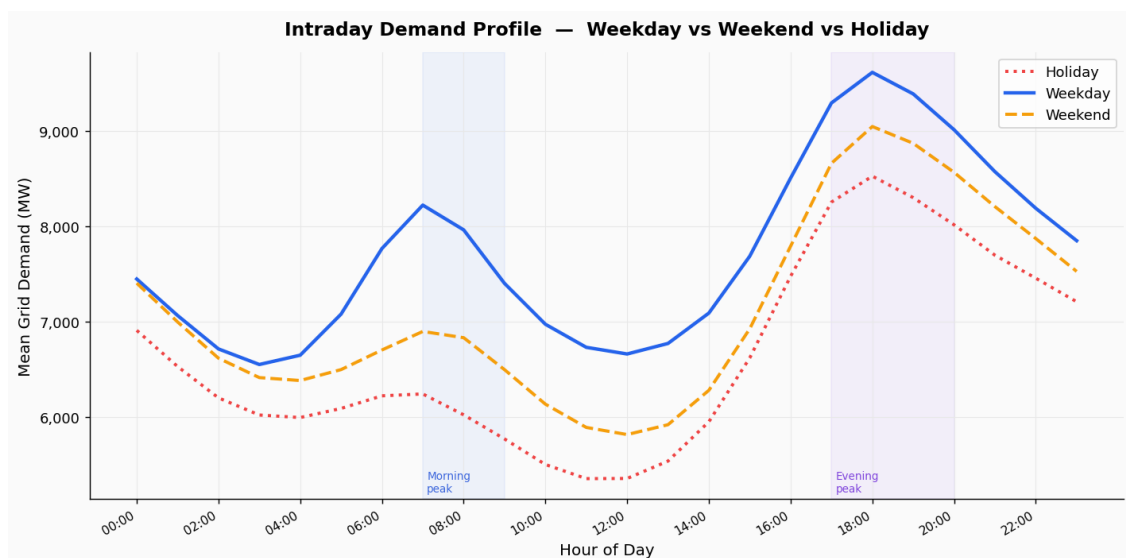


Figure 6. Mean NSW grid electricity demand (MW) by hour of day for weekdays (solid blue), weekends (dashed orange), and public holidays (dotted red). Weekdays show a morning peak at 8 AM (~8,200 MW) and an evening peak at 7 PM (~9,500 MW). Weekend and holiday profiles are 1,000-1,500 MW lower with no commercial morning peak.

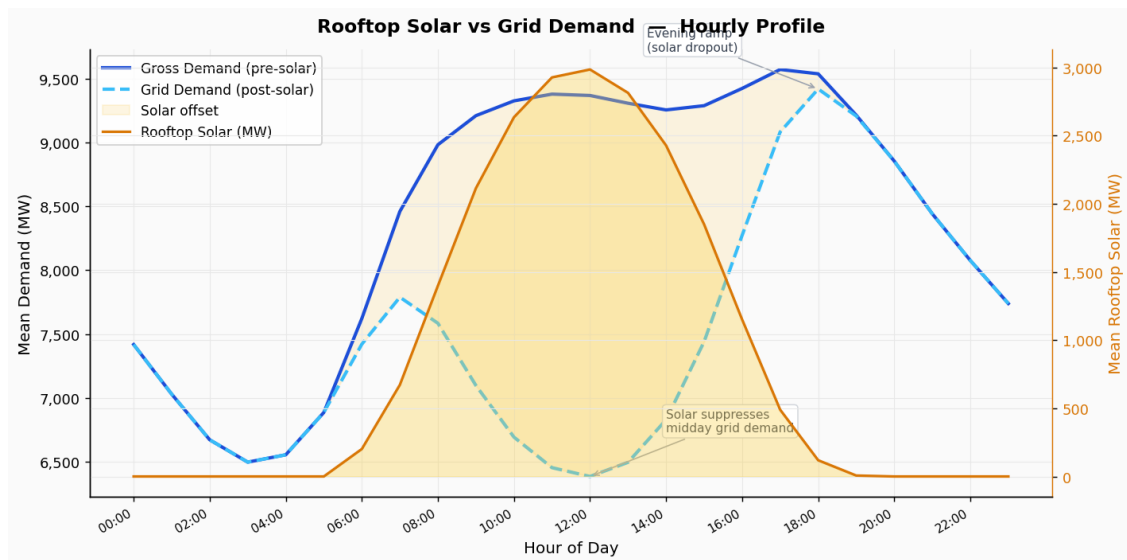


Figure 7. Mean hourly rooftop solar generation (MW, right axis, orange) overlaid on gross pre-solar demand (solid blue) and net post-solar grid demand (dashed teal). Solar peaks near midday (~3,000 MW), suppressing grid demand and creating the sharp evening ramp as generation drops — the “duck curve” pattern.

B Modelling Code

B.1 Data Preparation

```

1  import pandas as pd
2  import os
3
4  ## IMPORTING THE AEMO DATA
5
6  # relative directory path
7  directory = './data/demand'
8  dfs = []
9
10 for filename in os.listdir(directory):
11     if filename.endswith('.csv'):
12         # Create the full file path and read the file
13         file_path = os.path.join(directory, filename)
14         df_temp = pd.read_csv(file_path)
15
16         # FIX THE FIRST ROW IN THE LOOP
17         df_temp['SETTLEMENTDATE'] = pd.to_datetime(df_temp['SETTLEMENTDATE'])
18         if not df_temp.empty:
19             df_temp.loc[0, 'SETTLEMENTDATE'] -= pd.Timedelta(minutes=5)
20
21         # Check if column exists to prevent errors
22         print(f"File: {filename}")
23         dfs.append(df_temp)
24
25 df = pd.concat(dfs, ignore_index=True)
26 df.drop(columns=['REGION', 'PERIODTYPE'], inplace=True)
27 df.rename(columns={'SETTLEMENTDATE': 'DATE_TIME', 'TOTALDEMAND': 'TOTAL_DEMAND'},
28           → inplace=True, errors='ignore')
29
30 df['DATE_TIME'] = pd.to_datetime(df['DATE_TIME'])
31 df = df[df['DATE_TIME'].dt.minute.isin([0, 30])].copy()
32 df = df.sort_values('DATE_TIME').reset_index(drop=True)
33
34 ## CREATING COLUMNS FOR SCHOOL_HOLIDAY, PUBLIC_HOLIDAY
35 import holidays
36
37 # Convert to Pandas datetime for better parsing
38 df['DATE_TIME'] = pd.to_datetime(df['DATE_TIME'])
39
40 nsw_holidays = holidays.Australia(subdiv='NSW', years=range(2021, 2027))
41 df['PUBLIC_HOLIDAY'] = df['DATE_TIME'].dt.date.isin(nsw_holidays)
42
43 school_holiday_ranges = [
44     # 2022
45     ('2022-01-01', '2022-01-27'), # Summer 21-22 end
46     ('2022-04-11', '2022-04-22'), # Autumn
47     ('2022-07-04', '2022-07-15'), # Winter
48     ('2022-09-26', '2022-10-07'), # Spring
49     ('2022-12-21', '2022-12-31'), # Summer 22-23 start
50
51     # 2023
52     ('2023-01-01', '2023-01-26'), # Summer 22-23
53     ('2023-04-10', '2023-04-21'), # Autumn

```

```

53     ('2023-07-03', '2023-07-14'), # Winter
54     ('2023-09-25', '2023-10-06'), # Spring
55     ('2023-12-20', '2023-12-31'), # Summer 23-24 start
56
57     # 2024
58     ('2024-01-01', '2024-01-29'), # Summer 23-24 end
59     ('2024-04-15', '2024-04-26'), # Autumn
60     ('2024-07-08', '2024-07-19'), # Winter
61     ('2024-09-30', '2024-10-11'), # Spring
62     ('2023-12-23', '2023-12-31'), # Summer 24-25 start
63
64     # 2025
65     ('2025-01-01', '2025-01-30'), # Summer 24-25 end
66     ('2025-04-14', '2025-04-24'), # Autumn
67     ('2025-07-07', '2025-07-18'), # Winter
68     ('2025-09-29', '2025-10-10'), # Spring
69     ('2025-12-22', '2025-12-31'), # Summer 25-26 start
70
71     # 2026
72     ('2026-01-01', '2026-01-26'), # Summer 25-26 end
73     ('2026-04-07', '2026-04-17'), # Autumn
74     ('2026-07-06', '2026-07-17'), # Winter
75     ('2026-09-28', '2026-10-09'), # Spring
76     ('2026-12-18', '2026-12-31'), # Summer 26-27 start
77 ]
78
79 holiday_dates = set()
80 for start, end in school_holiday_ranges:
81     holiday_dates.update(pd.date_range(start, end).date)
82
83 df['SCHOOL_HOLIDAY'] = df['DATE_TIME'].dt.date.isin(holiday_dates) # check and store
84     ↪ boolean
85
86 ## WEATHER DATA
87 import pandas as pd
88 import datetime
89
90 def fetch_sydney_weather(start_year, end_year):
91     # Sydney Airport Station ID is usually 'YSSY' in the ASOS network
92     station = 'YSSY'
93     start_date = f"{start_year}-01-01+00:00"
94     end_date = f"{end_year}-12-31+23:59"
95
96     # Web scraping the global archive - added 'feel' and 'p01m' for transpiration
97     ↪ context
98     url = (f"https://mesonet.agron.iastate.edu/cgi-bin/request/asos.py?"
99           f"station={station}&data=tmpc&data=dwpc&data=relh&data=feel&data=sknt&"
100          f"data=skyc1&data=skyc2&data=skyc3&"
101          f"year1={start_year}&month1=1&day1=1&"
102          f"year2={end_year}&month2=12&day2=31&"
103          f"tz=Etc%2FGMT-10&format=onlycomma&latlon=no&missing=M")
104
105     print("Fetching weather data")
106     weather_df = pd.read_csv(url, skiprows=0, na_values='M')

```

```

107     # Rename columns for clarity
108     weather_df = weather_df.rename(columns={
109         'valid': 'DATE_TIME',
110         'tmpc': 'TEMPERATURE',
111         'relh': 'HUMIDITY',
112         'dwpc': 'DWPC',
113         'feel': 'APPARENT_TEMP',
114         'sknt': 'WIND_SPEED',
115         'skyc1': 'CLOUD_L1',
116         'skyc2': 'CLOUD_L2',
117         'skyc3': 'CLOUD_L3'
118     })
119
120     weather_df['DATE_TIME'] = pd.to_datetime(weather_df['DATE_TIME'])
121     weather_df['APPARENT_TEMP'] = ((weather_df['APPARENT_TEMP'] - 32) * 5/9) #
122         ↪ Fahrenheit to Celcius
123
124     weather_df['WIND_SPEED'] = (weather_df['WIND_SPEED'] * 1.852) # Convert Wind
125         ↪ Speed from knots to km/h (1 knot 1.852 km/h)
126
127     return weather_df
128
129 weather_df = fetch_sydney_weather(2022, 2026)
130
131 # Cleaning dataset
132 cloud_map = {'CLR': 0, 'SKC': 0, 'FEW': 2, 'SCT': 4, 'BKN': 6, 'OVC': 8}
133 for col in ['CLOUD_L1', 'CLOUD_L2', 'CLOUD_L3']:
134     weather_df[col] = weather_df[col].map(cloud_map).fillna(0).round()
135
136 weather_df['CLOUD_COVER'] = weather_df[['CLOUD_L1', 'CLOUD_L2', 'CLOUD_L3']].max(
137     ↪ axis=1)
138
139 weather_df.drop(['station', 'CLOUD_L1', 'CLOUD_L2', 'CLOUD_L3'], axis=1, inplace=
140     ↪ True, errors='ignore')
141
142 if 'DATE_TIME' in df.columns:
143     demand_df = df.set_index('DATE_TIME').sort_index()
144 else:
145     demand_df = df.sort_index()
146
147 if 'DATE_TIME' in weather_df.columns:
148     weather_df = weather_df.set_index('DATE_TIME').sort_index()
149 else:
150     weather_df = weather_df.sort_index()
151
152 weather_numeric = weather_df.select_dtypes(include=['number', 'float', 'int'])
153 weather_5min = weather_numeric.reindex(demand_df.index)
154
155 # Interpolate the gaps (Linear interpolation)
156 weather_5min = weather_5min.interpolate(method='linear')
157 weather_5min = weather_5min.bfill().ffill()
158
159 # merge back together
160 demand_df = demand_df.drop(columns=weather_5min.columns.intersection(demand_df.
161     ↪ columns))
162 df = pd.concat([demand_df, weather_5min], axis=1).reset_index()
163

```

```

58 if 'index' in df.columns:
59     df = df.rename(columns={'index': 'DATE_TIME'})
60
61
62 ### Consumer spending and confidence
63
64
65 ## RBA Monthly Cash Rate
66 cash_rate_df = pd.read_csv('./data/rba_monthly_cash_rate.csv')
67 df['Month_Year'] = df['DATE_TIME'].dt.strftime('%Y-%m') # parse date format like
    ↳ '2023-01'
68 df = df.merge(cash_rate_df, left_on='Month_Year', right_on='DATE', how='left') #
    ↳ broadcast join
69 df = df.drop(columns=['Month_Year', 'DATE']) # clean columns
70
71 ## Monthly Household Spending Indicator (MHSI)
72 spending_df = pd.read_csv('./data/spending_millions.csv')
73 df['join_key'] = df['DATE_TIME'].dt.strftime('%b-%y')
74
75 # Merge and clean up
76 df = df.merge(spending_df, left_on='join_key', right_on='Month', how='left')
77 df = df.drop(columns=['join_key', 'Month'])
78
79 # Consumer Confidence (Westpac)
80 consumer_conf = pd.read_csv('./data/consumer_conf.csv')
81 df['month_key'] = df['DATE_TIME'].dt.strftime('%Y-%m')
82 df = df.merge(consumer_conf, left_on='month_key', right_on='year-month', how='left')
83 df = df.drop(columns=['month_key', 'year-month'])
84
85 pop_df = pd.read_csv('./data/population.csv')
86 pop_df['DATE_TIME'] = pd.to_datetime(pop_df['DATE_TIME'])
87
88 df = pd.merge(df, pop_df, on='DATE_TIME', how='left')
89 df['NSW_POPULATION'] = df['NSW_POPULATION'].interpolate(method='linear').round().
    ↳ astype(int)
90
91 from nemosis import dynamic_data_compiler
92 import pandas as pd
93
94 # Define project range
95 start_time = '2022/01/01 00:00:00'
96 end_time = '2026/02/01 00:00:00'
97
98 solar_df = dynamic_data_compiler(
99     start_time=start_time,
100     end_time=end_time,
101     table_name='ROOFTOP_PV_ACTUAL',
102     raw_data_location='./data/cache' # This is the required positional argument
103 )
104
105 # Filter for NSW and clean up
106 solar_nsw = solar_df[solar_df['REGIONID'] == 'NSW1'].copy()
107
108 # AEMO ROOFTOP_PV_ACTUAL usually uses 'INTERVAL_DATETIME' or 'SETTLEMENTDATE'
109 date_col = 'INTERVAL_DATETIME' if 'INTERVAL_DATETIME' in solar_nsw.columns else '
    ↳ SETTLEMENTDATE'

```

```

210
211 solar_nsw = solar_nsw[[date_col, 'POWER']].rename(
212     columns={date_col: 'DATE_TIME', 'POWER': 'ROOFTOP_SOLAR_MW'}
213 )
214
215 solar_nsw['DATE_TIME'] = pd.to_datetime(solar_nsw['DATE_TIME'])
216
217 # Aggregate duplicates to prevent row-doubling
218 solar_nsw = solar_nsw.groupby('DATE_TIME')['ROOFTOP_SOLAR_MW'].mean().reset_index()
219
220 # Perform the merge
221 df = pd.merge(df, solar_nsw[['DATE_TIME', 'ROOFTOP_SOLAR_MW']], on='DATE_TIME', how=
    ↳ 'left')
222 df['ROOFTOP_SOLAR_MW'] = df['ROOFTOP_SOLAR_MW'].bfill(limit=1)
223
224 # Standard linear interpolation for any other gaps
225 df['ROOFTOP_SOLAR_MW'] = df['ROOFTOP_SOLAR_MW'].interpolate(method='linear')
226
227 # Zero out nighttime values
228 df.loc[(df['DATE_TIME'].dt.hour >= 20) | (df['DATE_TIME'].dt.hour <= 5), '
    ↳ ROOFTOP_SOLAR_MW'] = 0
229 df['ROOFTOP_SOLAR_MW'] = df['ROOFTOP_SOLAR_MW'].fillna(0)
230
231 df['GROSS_DEMAND_MW'] = df['TOTAL_DEMAND'] + df['ROOFTOP_SOLAR_MW']
232
233 # round and save
234 df = df.round(4)
235 df.to_csv('./data/dataset.csv', index=False)

```

B.2 Modelling

```

1  # # Preprocessing
2
3  ! pip install -r requirements.txt -q
4
5
6  import pandas as pd
7  import numpy as np
8
9  df = pd.read_csv('./data/dataset.csv')
10
11 df.head()
12
13 print(df.shape)
14 df.info()
15 df.describe()
16
17 df.head()
18
19 # ## Feature Engineering
20
21 df['DATE_TIME'] = pd.to_datetime(df['DATE_TIME'])
22
23 df['HOUR'] = df['DATE_TIME'].dt.hour
24 df['DAY_OF_WEEK'] = df['DATE_TIME'].dt.dayofweek
25 df['MIN_SIN'] = np.sin(2 * np.pi * df['DATE_TIME'].dt.minute / 60)
26 df['MIN_COS'] = np.cos(2 * np.pi * df['DATE_TIME'].dt.minute / 60)

```

```

27 df['IS_WEEKEND'] = df['DAY_OF_WEEK'].isin([5, 6]).astype(int)
28 df['MONTH'] = df['DATE_TIME'].dt.month
29
30 def get_season(month):
31     if month in [12, 1, 2]:
32         return 'Summer'
33     elif month in [3, 4, 5]:
34         return 'Autumn'
35     elif month in [6, 7, 8]:
36         return 'Winter'
37     else:
38         return 'Spring'
39
40 df['SEASON'] = df['MONTH'].apply(get_season)
41 df = pd.get_dummies(df, columns=['SEASON'], prefix='SEASON')
42
43 df.shape
44
45 import altair as alt
46 import pandas as pd
47
48 # Calculate the correlation matrix for numerical columns
49 corr_matrix = df.select_dtypes('number').corr().reset_index().melt(id_vars='index')
50 corr_matrix.columns = ['Variable 1', 'Variable 2', 'Correlation']
51
52 # Create the heatmap
53 base = alt.Chart(corr_matrix).encode(
54     x=alt.X('Variable 1:O', title=None),
55     y=alt.Y('Variable 2:O', title=None)
56 )
57
58 heatmap = base.mark_rect().encode(
59     color=alt.Color('Correlation:Q', scale=alt.Scale(scheme='redblue', domain=[-1,
60     ↪ 1]))
61 )
62
63 # Add text labels for the correlation coefficients
64 text = base.mark_text().encode(
65     text=alt.Text('Correlation:Q', format='.2f'),
66     color=alt.condition(
67         "abs(datum.Correlation) > 0.5", # Using a string expression instead
68         alt.value('white'),
69         alt.value('black')
70     )
71 )
72
73 chart = (heatmap + text).properties(
74     width=600,
75     height=600
76 )
77
78 chart.save('./figs/multicollinearity_chart.pdf')
79
80 display(chart)
81
82 ## Splitting the Dataset

```

```

82
83 from sklearn.model_selection import train_test_split
84
85
86 y_target = 'GROSS_DEMAND_MW' # set the target variable
87 # y_target = 'TOTAL_DEMAND' # set the target variable
88 # y_target = 'ROOFTOP_SOLAR_MW' # set the target variable
89
90 df = df.sort_values('DATE_TIME')
91 df.drop(columns=[
92     'TOTAL_DEMAND',
93     'ROOFTOP_SOLAR_MW'
94 ], inplace=True, errors='ignore')
95 df.drop(columns=['DATE_TIME'], inplace=True, errors='ignore')
96
97 n = len(df)
98 train_end = int(n * 0.6)
99 val_end = int(n * 0.8)
100
101 # X_train, X_temp = train_test_split(df, test_size=0.4, random_state=42)
102 # X_val, X_test = train_test_split(X_temp, test_size=0.5, random_state=42)
103
104 X_train = df.iloc[:train_end]
105 X_val = df.iloc[train_end:val_end]
106 X_test = df.iloc[val_end:]
107
108 def plot_demand_dist(df, title):
109     """
110     Generates a binned distribution chart for GROSS_DEMAND_MW.
111     """
112     return alt.Chart(df).mark_bar().encode(
113         alt.X("GROSS_DEMAND_MW:Q", bin=alt.Bin(maxbins=30), title="Gross Demand (MW)
114             ↪ "),
115         alt.Y("count()", title="Frequency"),
116         tooltip=["count()"],
117         color=alt.value('#0066ff')
118     ).properties(
119         width=250,
120         height=200,
121         title=title
122     )
123
124 # Execute for all datasets
125 chart_train = plot_demand_dist(X_train, "X_train Distribution")
126 chart_val = plot_demand_dist(X_val, "X_val Distribution")
127 chart_test = plot_demand_dist(X_test, "X_test Distribution")
128
129 # Display side-by-side
130 final_dist_chart = (chart_train | chart_val | chart_test)
131 final_dist_chart.save('./figs/dist_chart.png')
132 display(final_dist_chart)
133
134 y_train, y_val, y_test = (
135     X_train.pop(y_target),
136     X_val.pop(y_target),
137     X_test.pop(y_target),

```

```
37 )
38
39 df_names = ['X_train', 'X_val', 'X_test']
40
41
42 print(X_train.shape)
43 print(X_val.shape)
44 print(X_test.shape)
45
46
47 # ## Data Transformation
48
49
50 # ### One-hot Encoding (OHE)
51
52 from sklearn.preprocessing import OneHotEncoder
53
54 ohe_encoder = OneHotEncoder()
55
56 for name in df_names:
57     df_ = globals()[name]
58
59     globals()[name] = df_.copy()
60
61
62 # ### Feature Scaling
63
64 from sklearn.preprocessing import StandardScaler
65
66 scaler = StandardScaler()
67 numeric_cols = X_train.select_dtypes(include=['number']).columns
68
69 scaler.fit(X_train[numeric_cols])
70
71 for name in df_names:
72     df_ = globals()[name]
73
74     df_[numeric_cols] = scaler.transform(df_[numeric_cols])
75
76     for col in df_.columns:
77         if df_[col].dtype == 'bool':
78             df_[col] = df_[col].astype('category').cat.codes
79
80     globals()[name] = df_.copy()
81
82 from sklearn.linear_model import Ridge
83
84 lin_reg = Ridge(
85     alpha=1.0,
86     fit_intercept=True,
87     random_state=42
88 )
89
90 lin_reg.fit(X_train, y_train)
91
92 lin_train_preds = lin_reg.predict(X_train)
```



```

193 lin_val_preds = lin_reg.predict(X_val)
194 lin_test_preds = lin_reg.predict(X_test)
195
196 print(f"Model Intercept: {lin_reg.intercept_}")
197
198 # %%
199 from xgboost import XGBRegressor
200
201 xgb_model = XGBRegressor(
202     n_estimators=1000,
203     learning_rate=0.05,
204     max_depth=6,
205     random_state=42
206 )
207
208 xgb_model.fit(
209     X_train, y_train,
210     eval_set=[(X_val, y_val)],
211     verbose=False
212 )
213
214 xgb_train_preds = xgb_model.predict(X_train)
215 xgb_val_preds = xgb_model.predict(X_val)
216 xgb_test_preds = xgb_model.predict(X_test)
217
218 # %%
219 from catboost import CatBoostRegressor
220
221 cat_model = CatBoostRegressor(
222     iterations=1000,
223     learning_rate=0.05,
224     depth=6,
225     loss_function='RMSE',
226     random_state=42
227 )
228
229 cat_model.fit(
230     X_train, y_train,
231     eval_set=(X_val, y_val),
232     early_stopping_rounds=50,
233     verbose=False
234 )
235
236 cat_train_preds = cat_model.predict(X_train)
237 cat_val_preds = cat_model.predict(X_val)
238 cat_test_preds = cat_model.predict(X_test)
239
240 # %%
241 import pandas as pd
242 import numpy as np
243 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
244
245 def get_metrics_row(y_train, p_train, y_val, p_val, y_test, p_test, n_feat):
246     """Calculate MultiIndex row data for model evaluation."""
247     def score(y, p):
248         r2 = r2_score(y, p)

```

```

249     # Prevent division by zero if n_feat is too high relative to sample size
250     adj_r2 = 1 - (1 - r2) * (len(y) - 1) / max((len(y) - n_feat - 1), 1)
251
252     # Calculate MAPE with a small epsilon to avoid division by zero
253     mape = np.mean(np.abs((y - p) / np.maximum(np.abs(y), 1e-10))) * 100
254
255     return {
256         'RMSE (MW)': round(np.sqrt(mean_squared_error(y, p)), 6),
257         'MAE (MW)': round(mean_absolute_error(y, p), 6),
258         'MAPE (%)': round(mape, 6),
259         'Adj R2': round(adj_r2, 6)
260     }
261
262     tr, vl, ts = score(y_train, p_train), score(y_val, p_val), score(y_test, p_test)
263
264     return {
265         ('RMSE (MW)', 'Train'): tr['RMSE (MW)'], ('RMSE (MW)', 'Val'): vl['RMSE (MW)']
266         ↪ ], ('RMSE (MW)', 'Test'): ts['RMSE (MW)'],
267         ('MAE (MW)', 'Train'): tr['MAE (MW)'], ('MAE (MW)', 'Val'): vl['MAE (MW)']
268         ↪ ], ('MAE (MW)', 'Test'): ts['MAE (MW)'],
269         ('MAPE (%)', 'Train'): tr['MAPE (%)'], ('MAPE (%)', 'Val'): vl['MAPE (%)']
270         ↪ ], ('MAPE (%)', 'Test'): ts['MAPE (%)'],
271         ('Adj R2', 'Train'): tr['Adj R2'], ('Adj R2', 'Val'): vl['Adj R2'],
272         ↪ ('Adj R2', 'Test'): ts['Adj R2']
273     }
274
275     n_feat = X_train.shape[1]
276
277     # Data aggregation - Added n_feat to each call
278     data = {
279         'Linear Regression': get_metrics_row(y_train, lin_train_preds, y_val,
280         ↪ lin_val_preds, y_test, lin_test_preds, n_feat),
281         'XGBoost': get_metrics_row(y_train, xgb_train_preds, y_val, xgb_val_preds,
282         ↪ y_test, xgb_test_preds, n_feat),
283         'CatBoost': get_metrics_row(y_train, cat_train_preds, y_val, cat_val_preds,
284         ↪ y_test, cat_test_preds, n_feat)
285     }
286
287     # DataFrame construction
288     metrics_df = pd.DataFrame.from_dict(data, orient='index')
289     metrics_df.columns = pd.MultiIndex.from_tuples(metrics_df.columns)
290
291     # Styling for display
292     styled_df = metrics_df.style.set_table_styles([
293         {'selector': 'th', 'props': [('text-align', 'center'), ('background-color', "
294         ↪ #0077FF"), ('color', 'white')]},
295         {'selector': 'td', 'props': [('text-align', 'center')]}
296     ]).set_properties(**{'text-align': 'center'})
297
298     display(styled_df)
299
300     # %%
301     from sklearn.linear_model import Ridge
302     from sklearn.model_selection import GridSearchCV
303
304     # Define parameter grid for Linear Model (Ridge)

```

```

297 # alpha is the regularization strength
298 linear_param_grid = {
299     'alpha': [0.1, 1.0, 10.0, 100.0],
300     'fit_intercept': [True, False]
301 }
302
303 # Create GridSearchCV for Linear Model
304 linear_grid_search = GridSearchCV(
305     Ridge(),
306     linear_param_grid,
307     cv=3,
308     scoring='neg_mean_squared_error',
309     n_jobs=-1
310 )
311
312 # Fit the grid search
313 linear_grid_search.fit(X_train, y_train)
314
315 # Get best model and parameters
316 best_linear_model = linear_grid_search.best_estimator_
317 print(f"Best Linear parameters: {linear_grid_search.best_params_}")
318 print(f"Best CV score (neg_mse): {linear_grid_search.best_score_}")
319
320 # Make predictions with best model
321 linear_grid_train_preds = best_linear_model.predict(X_train)
322 linear_grid_val_preds = best_linear_model.predict(X_val)
323 linear_grid_test_preds = best_linear_model.predict(X_test)
324
325 # %%
326 from sklearn.model_selection import GridSearchCV
327
328 # Define parameter grid for CatBoost
329 param_grid = {
330     'depth': [x for x in range(4, 9)],
331     'learning_rate': [0.01, 0.05, 0.1],
332     'iterations': [500, 1000, 1500]
333 }
334
335 # Create GridSearchCV for CatBoost
336 cat_grid_search = GridSearchCV(
337     CatBoostRegressor(loss_function='RMSE', random_state=42, verbose=False),
338     param_grid,
339     cv=3,
340     scoring='neg_mean_squared_error',
341     n_jobs=-1
342 )
343
344 # Fit the grid search
345 cat_grid_search.fit(X_train, y_train)
346
347 # Get best model and parameters
348 best_cat_model = cat_grid_search.best_estimator_
349 print(f"Best CatBoost parameters: {cat_grid_search.best_params_}")
350 print(f"Best CV score (neg_mse): {cat_grid_search.best_score_}")
351
352 # Make predictions with best model

```

```

353 cat_grid_train_preds = best_cat_model.predict(X_train)
354 cat_grid_val_preds = best_cat_model.predict(X_val)
355 cat_grid_test_preds = best_cat_model.predict(X_test)
356
357 from sklearn.model_selection import GridSearchCV
358
359 # Define parameter grid for XGBoost
360 xgb_param_grid = {
361     'max_depth': [x for x in range(4, 9)],
362     'learning_rate': [0.01, 0.05, 0.1],
363     'n_estimators': [500, 1000, 1500]
364 }
365
366 # Create GridSearchCV for XGBoost
367 xgb_grid_search = GridSearchCV(
368     xgb_model.__class__(random_state=42),
369     xgb_param_grid,
370     cv=3,
371     scoring='neg_mean_squared_error',
372     n_jobs=-1
373 )
374
375 # Fit the grid search
376 xgb_grid_search.fit(X_train, y_train)
377
378 # Get best model and parameters
379 best_xgb_model = xgb_grid_search.best_estimator_
380 print(f"Best XGBoost parameters: {xgb_grid_search.best_params_}")
381 print(f"Best CV score (neg_mse): {xgb_grid_search.best_score_}")
382
383 # Make predictions with best model
384 xgb_grid_train_preds = best_xgb_model.predict(X_train)
385 xgb_grid_val_preds = best_xgb_model.predict(X_val)
386 xgb_grid_test_preds = best_xgb_model.predict(X_test)
387
388 import altair as alt
389 import pandas as pd
390 import numpy as np
391 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
392
393 # Assuming X_train is the feature matrix
394 n_feat = X_train.shape[1]
395
396 # Data aggregation - Added n_feat to each call
397 data = {
398     "Linear Regression": get_metrics_row(
399         y_train,
400         linear_grid_train_preds,
401         y_val,
402         linear_grid_val_preds,
403         y_test,
404         linear_grid_test_preds,
405         n_feat,
406     ),
407     "XGBoost": get_metrics_row(
408         y_train,

```

```

409     xgb_grid_train_preds,
410     y_val,
411     xgb_grid_val_preds,
412     y_test,
413     xgb_grid_test_preds,
414     n_feat,
415 ),
416 "CatBoost": get_metrics_row(
417     y_train,
418     cat_grid_train_preds,
419     y_val,
420     cat_grid_val_preds,
421     y_test,
422     cat_grid_test_preds,
423     n_feat,
424 ),
425 }
426
427 # DataFrame construction
428 metrics_df = pd.DataFrame.from_dict(data, orient="index")
429 metrics_df.columns = pd.MultiIndex.from_tuples(metrics_df.columns)
430
431 # Styling for display
432 styled_df = metrics_df.style.set_table_styles(
433     [
434         {
435             "selector": "th",
436             "props": [
437                 ("text-align", "center"),
438                 ("background-color", "#0077FF"),
439                 ("color", "white"),
440             ],
441         },
442         {"selector": "td", "props": [("text-align", "center")]}],
443     )
444 ).set_properties(**{"text-align": "center"})
445
446 display(styled_df)
447
448 ## Regression plots
449 alt.data_transformers.enable("vegafusion") # for large plots
450
451 plot_data = pd.DataFrame({
452     'Actual': y_test.values,
453     'Linear Regression': linear_grid_test_preds,
454     'XGBoost': xgb_grid_test_preds,
455     'CatBoost': cat_grid_test_preds
456 }).reset_index()
457
458 melted_df = plot_data.melt(id_vars=['index', 'Actual'],
459                             var_name='Model',
460                             value_name='Prediction')
461
462 actual_line = alt.Chart(melted_df).mark_line(color='black', strokeWidth=2, opacity
463     ↪ =0.5).encode(
464     x=alt.X('index:Q', title='Sample Index'),

```

```

464     y=alt.Y('Actual:Q', title='MW'),
465 )
466
467 model_order = ['Linear Regression', 'XGBoost', 'CatBoost']
468
469 pred_line = alt.Chart(melted_df).mark_line(strokeDash=[5, 5]).encode(
470     x='index:Q',
471     y='Prediction:Q',
472     color=alt.Color('Model:N',
473                     scale=alt.Scale(scheme='tableau10'),
474                     sort=model_order) # disable sorting
475 )
476
477 final_chart = alt.layer(actual_line, pred_line).properties(
478     width=700,
479     height=200
480 ).facet(
481     row=alt.Row('Model:N', sort=model_order) # disable sorting
482 ).resolve_scale(
483     y='shared'
484 )
485
486 display(final_chart)
487
488
489 import altair as alt
490 import pandas as pd
491 import numpy as np
492
493 def plot_feature_importance(model, model_name, columns, save_path=None):
494     g_color = '#058af7'
495
496     # Extract importance based on model type
497     if hasattr(model, 'coef_'):
498         importance = np.abs(model.coef_)
499         x_title = 'Contribution Score (Abs Coeff)'
500     elif hasattr(model, 'get_feature_importance'):
501         importance = model.get_feature_importance()
502         x_title = 'Contribution Score'
503     elif hasattr(model, 'feature_importances_'):
504         importance = model.feature_importances_
505         x_title = 'Contribution Score'
506     else:
507         raise ValueError("Model type not supported for importance extraction.")
508
509     df = pd.DataFrame({
510         'Feature': columns,
511         'Importance': importance
512     }).sort_values(by='Importance', ascending=False)
513
514     chart = alt.Chart(df).mark_bar(color=g_color).encode(
515         x=alt.X('Importance:Q', title=x_title),
516         y=alt.Y('Feature:N', sort='-x', title='Predictor Variable'),
517         tooltip=['Feature', 'Importance']
518     ).properties(
519         title=f'Key Drivers of Energy Demand ({model_name})',

```

```

520         width=600,
521         height=400
522     )
523
524     if save_path:
525         chart.save(save_path)
526
527     return chart
528
529 ## Ridge (Linear Regression)
530 lin_importance_chart = plot_feature_importance(best_linear_model, 'Ridge Linear',
531     ↪ X_train.columns, './figs/lin_imp_chart.png')
532
533 ## CatBoost
534 cat_importance_chart = plot_feature_importance(best_cat_model, 'CatBoost', X_train.
535     ↪ columns, './figs/xgb_imp_chart.png')
536
537 ## XGBoost
538 xgb_importance_chart = plot_feature_importance(best_xgb_model, 'XGBoost', X_train.
539     ↪ columns, './figs/cat_imp_chart.png')
540
541 display(lin_importance_chart)
542 display(cat_importance_chart)
543 display(xgb_importance_chart)
544
545 with pd.option_context('display.max_columns', None):
546     display(X_train.head())
547
548 import pandas as pd
549 import numpy as np
550 import altair as alt
551 import scipy.stats as stats
552
553 def plot_qq_grid(models_dict):
554     """
555     models_dict: {
556         'Linear': {'Train': y_p_train, 'Val': y_p_val, 'Test': y_p_test},
557         'XGBoost': { ... },
558         'CatBoost': { ... }
559     }
560     """
561     all_data = []
562
563     # Nested loop to build the 'Tidy' dataframe
564     for model_name, splits in models_dict.items():
565         for split_name, y_pred in splits.items():
566             # Match the correct true values
567             y_true = {'Train': y_train, 'Val': y_val, 'Test': y_test}[split_name]
568
569             # Calculate Standardized Residuals
570             res = (y_true - y_pred).values
571             res_std = (res - np.mean(res)) / np.std(res)
572             res_sorted = np.sort(res_std)
573
574             # Theoretical Normal Quantiles

```

```

$73     theoretical = stats.norm.ppf(np.linspace(0.01, 0.99, len(res_sorted)))
$74
$75     tmp_df = pd.DataFrame({
$76         'Theoretical': theoretical,
$77         'Sample': res_sorted,
$78         'Model': model_name,
$79         'Split': split_name
$80     })
$81
$82     # Downsample for browser performance
$83     if len(tmp_df) > 3000:
$84         tmp_df = tmp_df.sample(3000, random_state=42).sort_values('
            ↳ Theoretical')
$85
$86     all_data.append(tmp_df)
$87
$88 df = pd.concat(all_data)
$89
$90 # Plot points
$91 points = alt.Chart(df).mark_circle(size=20, opacity=0.4).encode(
$92     x=alt.X('Theoretical:Q', title='Theoretical'),
$93     y=alt.Y('Sample:Q', title='Residuals'),
$94     color=alt.Color('Split:N', legend=None)
$95 ).properties(width=200, height=200)
$96
$97 # The Reference Line (Standard Normal y=x)
$98 line_df = pd.DataFrame({'x': [-3, 3], 'y': [-3, 3]})
$99 line = alt.Chart(line_df).mark_line(color='red', strokeDash=[4,4]).encode(
$100     x='x:Q', y='y:Q'
$101 )
$102
$103 # Combine and facet into 3x3
$104 grid = alt.layer(points, line, data=df).facet(
$105     column=alt.Column('Split:N', sort=['Train', 'Val', 'Test']),
$106     row=alt.Row('Model:N')
$107 ).resolve_scale(
$108     x='shared', y='shared'
$109 )
$110
$111 return grid
$112
$113 results_grid = {
$114     'Linear': {'Train': best_linear_model.predict(X_train), 'Val': best_linear_model
            ↳ .predict(X_val), 'Test': best_linear_model.predict(X_test)},
$115     'XGBoost': {'Train': best_xgb_model.predict(X_train), 'Val': best_xgb_model.
            ↳ predict(X_val), 'Test': best_xgb_model.predict(X_test)},
$116     'CatBoost': {'Train': best_cat_model.predict(X_train), 'Val': best_cat_model.
            ↳ predict(X_val), 'Test': best_cat_model.predict(X_test)}
$117 }
$118 plot_qq_grid(results_grid)

```